Store Advisor

Data Cleaning System — Complete MVB Architecture & Implementation Specification

Scope: Basic Pipeline + Advanced Pipeline + Agent Architecture

Current implementation target: MVB (Minimum Viable Build)

This document defines the complete design of the data-cleaning subsystem: responsibilities, architecture, profiling, cleaning operations, Basic and Advanced pipelines, the future Agent pipeline, APIs, validation, reporting, project structure, and an implementation roadmap.

Version 1.0 — August 2026

1. Executive Summary

Store Advisor should not treat data cleaning as one large function. The system should be built as reusable cleaning operations plus separate pipeline strategies. The MVB implements two deterministic/rule-based pipelines: Basic and Advanced. The Agent pipeline is designed as a future extension in which an intelligent planner selects operations based on the dataset profile and user/business requirements.

> The key architectural rule is: operations are reusable; pipelines decide when to use them. This prevents the Agent from requiring a rewrite of the Basic and Advanced implementations.

2. Goals and Non-Goals

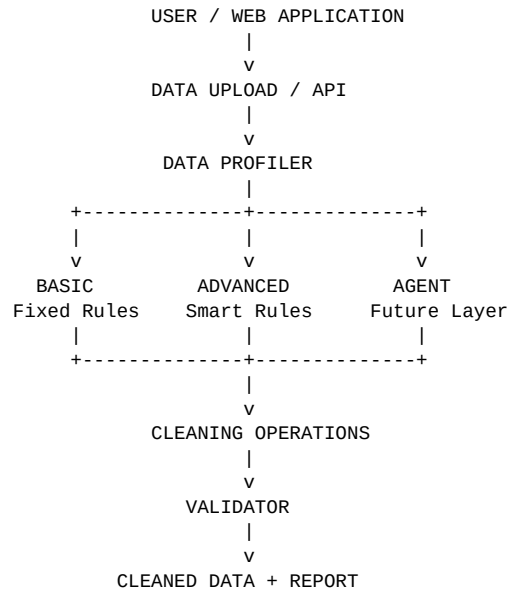
Goals

- Load common tabular data formats.
- Profile the dataset before cleaning.
- Detect duplicates, missing values, data types, cardinality, and numerical distributions.
- Provide a predictable Basic pipeline.
- Provide a more sophisticated rule-based Advanced pipeline.
- Produce cleaning logs and validation results.
- Expose a stable backend interface that the website can call.

Non-goals for the MVB

- Do not depend on an LLM for Basic or Advanced cleaning.
- Do not automatically make business-semantic decisions that require domain knowledge.
- Do not blindly delete every statistical outlier.
- Do not let an LLM directly mutate the DataFrame without controlled tools and validation.

3. System Architecture



The profiler is shared by all pipelines. The operations layer is shared as much as possible. The pipeline layer contains the strategy. The validator checks whether the resulting dataset is structurally valid and whether cleaning introduced obvious failures.

4. Three Pipeline Model

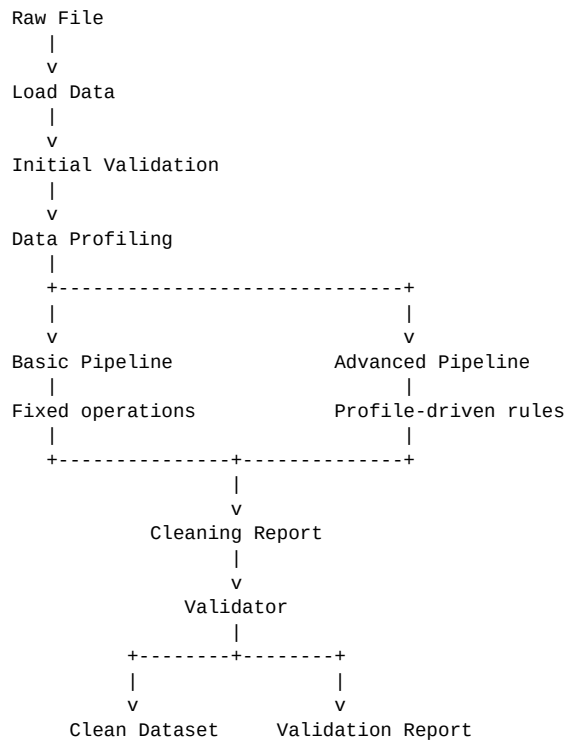
Pipeline	Purpose	Decision Logic	MVB?
Basic	Fast standard cleaning	Fixed predefined rules	Yes
Advanced	More context-aware cleaning	Rule-based decisions from profiling	Yes
Agent	Adaptive intelligent cleaning	Planner + rules/LLM + tools	Future

Basic pipeline: the user chooses Basic and the system executes the same predefined sequence every time.

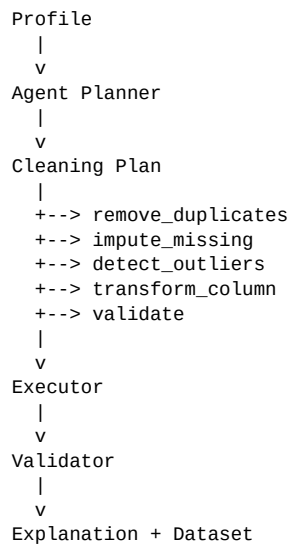
Advanced pipeline: the user chooses Advanced and the system analyzes the dataset first, then applies more selective rules.

Agent pipeline: the website can eventually expose a Smart Agent option. The Agent will create a plan, execute only approved tools, validate the result, and explain decisions.

5. End-to-End Data Flow



A future Agent adds a planning layer between profiling and execution:



6. Data Profiling Specification

The profiler should produce machine-readable metadata rather than only printing to the console.

Category	Fields
Dataset	rows, columns, duplicate_rows
Column	name, dtype, missing_count, missing_percentage, unique_count
Numerical	mean, median, min, max, optional quantiles
Categorical	unique count, optional top values/frequencies
Quality	constant column, high missingness, possible identifier

7. Reusable Cleaning Operations

Operations should be small, testable functions. Examples:

- remove_duplicates(df)
- handle_missing_values_basic(df)
- handle_missing_values_advanced(df, profile)
- detect_outliers_iqr(df, column)
- remove_outliers_iqr(df, column)
- cap_outliers(df, column)
- drop_column(df, column)
- fix_data_types(df, rules)
- validate_dataset(df)

> A pipeline composes these operations. An operation should not decide the whole strategy of the application.

8. Basic Pipeline — MVB

```
def basic_pipeline(df):
    df = df.copy()

    # 1. Remove exact duplicate rows
    df = remove_duplicates(df)

    # 2. Fill missing values
    # Numerical -> median
    # Categorical -> mode
    df = handle_missing_values_basic(df)

    # 3. Apply a predefined numerical outlier rule
    for column in df.select_dtypes(include=np.number).columns:
        df = remove_outliers_iqr(df, column)

    return df
```

Basic rules should be documented and stable. It is acceptable for the Basic pipeline to be intentionally simple; its value is speed, predictability, and ease of use.

> Important restriction: never run IQR outlier logic over categorical columns, text columns, or arbitrary identifiers.

9. Advanced Pipeline — MVB

```
def advanced_pipeline(df):
    df = df.copy()

    profile = profile_dataset(df)

    df = remove_duplicates(df)

    df = handle_missing_values_advanced(
        df, profile
    )

    df = handle_outliers_advanced(
        df, profile
    )

    return df
```

The Advanced pipeline is still rule-based. It is not an Agent. The difference is that its decisions depend on measurable dataset characteristics.

10. Advanced Decision Rules

Condition	Example action
Missing > 50%	Recommend/drop the column rather than impute blindly
Numerical missing values	Median or another statistically justified method
Categorical missing values	Mode or explicit Unknown category
Numerical cardinality < 10	Be cautious with outlier removal; may be discrete/category-like
Outliers < 5%	Potential removal can be considered
Outliers 5–15%	Consider capping/winsorization rather than deleting
Outliers > 15%	Keep and flag; investigate distribution/business meaning
Constant column	Flag for possible removal

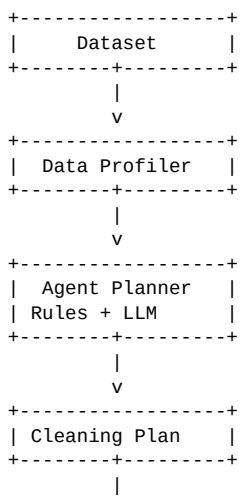
These are MVB engineering heuristics, not universal truths. The production system should eventually allow domain-aware rules and user overrides.

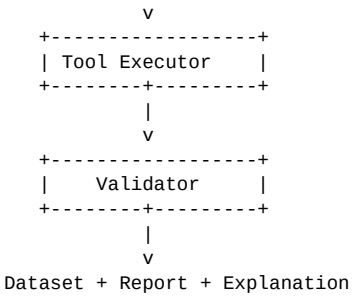
11. Agent Pipeline — Future Architecture

> The Agent should not directly manipulate a DataFrame using arbitrary generated code. It should select from a controlled set of tools.

```
class DataCleaningAgent:

    def run(self, df, user_preferences=None):
        profile = profile_dataset(df)

        plan = self.create_plan(
            profile,
            user_preferences
        )

        result = self.execute_plan(
            df,
            plan
        )

        validation = validate_dataset(
            result
        )

        return {
            "data": result,
            "plan": plan,
            "validation": validation
        }
```

12. Agent Internal Architecture





The planner should output structured actions such as remove_duplicates, median_imputation, mode_imputation, cap_outliers, drop_column, and so on. The executor validates action parameters before applying them.

13. Website / Backend Contract

The frontend should not need to know the internal cleaning implementation. It sends a pipeline selection and receives results.

```
POST /api/clean

{
    "pipeline": "basic",
    "file_id": "abc123"
}
```

For Advanced:

```
{
    "pipeline": "advanced",
    "file_id": "abc123"
}
```

Future Agent:

```
{
    "pipeline": "agent",
    "file_id": "abc123",
    "preferences": {
        "preserve_rows": true,
        "allow_column_drop": false
    }
}
```

A successful response should contain both the cleaned dataset reference and an auditable report:

```
{
    "status": "success",
    "pipeline": "advanced",
    "original_shape": [10000, 12],
    "cleaned_shape": [9820, 12],
    "operations": [
        {
            "action": "remove_duplicates",
            "affected_rows": 180
        }
    ],
    "validation": {
        "missing_values": 0,
        "duplicate_rows": 0
    }
}
```

14. Recommended Project Structure

```
store-advisor/
|
+-- backend/
|   |
|   +-- app/
|       |
|       +-- api/
|       |   +-- cleaning.py
|       |
|       +-- services/
|       |   +-- profiler.py
|       |   +-- validator.py
|       |
|       +-- cleaning/
|       |   +-- operations/
|       |   |   +-- duplicates.py
|       |   |   +-- missing_values.py
|       |   |   +-- outliers.py
|       |   |   +-- types.py
|       |   |
|       |   +-- pipelines/
|       |       +-- basic.py
|       |       +-- advanced.py
|       |       +-- agent.py
|       |
|       +-- schemas/
```

```
|       |   +-- cleaning.py
|       |
|       +-- main.py
|
+-- tests/
|   +-- test_profiler.py
|   +-- test_operations.py
|   +-- test_basic_pipeline.py
|   +-- test_advanced_pipeline.py
|
+-- frontend/
    +-- ...
```

15. Clean Pipeline Interface

Keep one entry point for the application:

```
def run_pipeline(df, pipeline="basic"):

    if pipeline == "basic":
        return basic_pipeline(df)

    if pipeline == "advanced":
        return advanced_pipeline(df)

    if pipeline == "agent":
        return agent_pipeline(df)

    raise ValueError(
        "Invalid pipeline"
    )
```

For the MVB, the Agent can remain disabled while the interface already exists. This prevents the website contract from changing later.

16. Validation Requirements

- Dataset must not become empty unless explicitly intended.
- Column count must not unexpectedly become zero.
- Duplicate count should be reported after cleaning.
- Missing-value count should be reported after cleaning.
- Data types should be compared before and after cleaning.
- Number of removed rows should be recorded.
- Dropped columns should be recorded with reasons.
- Outlier actions should state whether values were removed, capped, or retained.
- Every operation should be auditable.

17. Cleaning Report

The report is a first-class output, not just console text. The website can use it to display a summary such as:

```
Cleaning completed

Pipeline: Advanced

Rows:
  Before: 10,000
  After : 9,820

Duplicates:
  Removed: 180

Missing values:
  age      -> median
  city     -> mode

Outliers:
  salary   -> capped
  quantity -> retained

Validation:
  Status: PASS
```

18. Error Handling

- Unsupported file type -> clear API error.
- Malformed file -> loading error with user-safe message.
- Empty dataset -> validation failure.
- Invalid pipeline name -> 400-level API error.
- Unsupported operation parameter -> reject the action.
- Unexpected cleaning failure -> do not silently return corrupted data.

19. Testing Strategy

Create small synthetic datasets for each behavior.

```
def test_remove_duplicates():
    df = pd.DataFrame({
        "age": [20, 20, 30]
    })

    result = remove_duplicates(df)

    assert len(result) == 2


def test_missing_numeric():
    df = pd.DataFrame({
        "age": [20, np.nan, 30]
    })

    result = handle_missing_values_basic(df)

    assert result["age"].isna().sum() == 0
```

Test cases should include:

- No missing values.
- All values missing in one column.
- High missingness.
- Duplicate rows.
- No numerical columns.
- Only numerical columns.
- Only categorical columns.
- Identifier-like numerical columns.
- Extreme outliers.
- Empty dataset.
- Very small datasets.

20. Important Design Corrections to Avoid

- Do not apply IQR to every column.
- Do not return a Series where the pipeline expects a DataFrame.
- Do not use KNN imputation blindly on categorical data.
- Do not drop outliers simply because they are statistically unusual.
- Do not let the Agent execute arbitrary Python generated by an LLM.
- Do not hide cleaning decisions from the user.
- Do not mix frontend logic with cleaning logic.

21. MVB Implementation Order

```
PHASE 1
-------
1. Load CSV/XLSX
2. Basic profiler
3. Duplicate operation
4. Missing-value operation
5. Basic outlier operation
6. Basic pipeline
```

7. Validation
8. Tests

PHASE 2
----
1. Stronger profiling
2. Advanced missing-value rules
3. Advanced outlier rules
4. More data-type checks
5. Cleaning report
6. Advanced pipeline
7. API endpoint
8. Frontend pipeline selection

PHASE 3 — FUTURE
----
1. Agent planner
2. Structured cleaning plan
3. Controlled tools
4. LLM integration
5. User preferences
6. Validation loop
7. Explanation generation
8. Human approval for risky operations

22. Future Agent Decision Flow

The future Smart Agent should make decisions at the strategy level, not invent low-level code.

```
User
 |
 | "Clean this dataset"
 v
Profiler
 |
 | profile JSON
 v
Planner
 |
 +--> Is missingness significant?
 |
 +--> Is this column likely an ID?
 |
 +--> Are outliers sparse or widespread?
 |
 +--> Should rows be preserved?
 |
 +--> Is dropping a column acceptable?
 |
 v
Structured Plan
 |
 v
Approval / Policy Check
 |
 v
Tool Executor
 |
 v
Validator
 |
 +--> PASS -> Final result
 |
 +--> FAIL -> Re-plan / rollback
```

23. Agent Safety and Control

- Every action should come from an allow-list of tools.
- High-impact actions such as dropping columns or many rows should be flagged.
- The Agent should have access to profiling results, not unrestricted system access.
- Use a maximum number of planning/execution steps.
- Keep the original dataset immutable so operations can be rolled back.
- Store a complete action log.
- Validate after every major operation or at least after the complete plan.

24. Why This Architecture Fits Store Advisor

Store Advisor is intended to progress from deterministic data engineering toward intelligent business/data assistance. Separating operations from pipelines gives the project a clean evolution path:

```
MVB
 |
 +-- Basic
 |     Fixed, predictable
 |
 +-- Advanced
       Profile-driven rules

Future
 |
 +-- Agent
       Planning + controlled tools
       + validation + explanation
```

```
|
+-- EDA
+-- ML preparation
+-- Business analysis
```

The same principle can later be reused for EDA and ML preparation: reusable tools underneath, pipeline/agent strategy above them.

25. Reference Implementation Checklist

• Create profiler.py.

• Create cleaning operations as independent modules.

• Create basic.py and advanced.py.

• Create validator.py.

• Create a common run_pipeline() interface.

• Write unit tests before integrating the frontend.

• Return structured reports from the backend.

• Keep original data unchanged.

• Add logging for every transformation.

• Expose only Basic and Advanced in the MVB UI.

• Keep Agent behind a feature flag until the planner is stable.

26. Final Recommended Mental Model

Operation = How to perform one action.

Pipeline = Which actions to perform and in what order.

Profiler = What is happening in the dataset.

Advanced rules = How to choose actions from measurable conditions.

Agent = How to plan actions dynamically from data, policies, and eventually an LLM.

Validator = Whether the result is acceptable.

Report = What happened and why.

> Final recommendation: implement Basic and Advanced cleanly first. Do not rush the LLM Agent. If the operation layer, profiler, validation, logging, and pipeline interface are solid, the Agent becomes an additional decision-making layer instead of a complete rewrite.